

Ruzino曲面参数化实验报告

宁尚哲 PB24000099

April 9, 2026

Abstract

本文实现了基于Tutte参数化的曲面参数化算法，包括均匀权重、Cotangent权重和Floater权重的实现。通过对半边数据结构的操作，实现了边界检测、边界映射到平面区域以及拉普拉斯方程的求解。实验结果表明，不同权重策略对参数化质量有显著影响，其中Floater权重能够提供更好的形状保持性。

1 数学原理

1.1 Tutte参数化理论基础

Tutte参数化是一种经典的曲面参数化方法，基于图论中的Tutte嵌入理论。对于一个单连通的三角化平面图 $G(V, E, F)$ ，其中 V 为顶点集合， E 为边集合， F 为面集合，该方法通过以下方式实现参数化：

对于内部顶点 i ，其位置 $\mathbf{v}_i$ 通过其邻域顶点的凸组合确定：

$$\mathbf{v}_i = \sum_{j \in N(i)} w_j \mathbf{v}_j \quad (1)$$

其中 $N(i)$ 表示顶点 i 的1-邻域， w_j 为相应的权重系数，满足 $\sum_{j \in N(i)} w_j = 1$ 。

1.2 不同权重策略

1.2.1 均匀权重 (Uniform Weights)

最简单的权重策略，每个邻域顶点分配相等权重：

$$w_j = \frac{1}{d_i} \quad (2)$$

其中 d_i 为顶点 i 的度数。

1.2.2 Cotangent权重

基于原网格几何的权重策略，利用三角形的内角信息：

$$w_{ij} = \cot \alpha_{ij} + \cot \beta_{ij} \quad (3)$$

其中 α_{ij} 和 β_{ij} 分别是共享边 (i, j) 的两个三角形中对面的角度。

1.2.3 Floater权重 (形状保持权重)

采用平均值坐标的权重策略，具有良好的形状保持性质：

$$w_k = \frac{\tan(\alpha_k/2) + \tan(\beta_k/2)}{|\mathbf{v}_k - \mathbf{v}_i|} \quad (4)$$

其中 α_k 和 β_k 是相邻顶点 k 处的角度。

1.3 拉普拉斯方程与极小曲面

对于极小曲面问题，目标是使拉普拉斯算子作用于顶点位置为零：

$$\frac{1}{d_i} \sum_{j \in N(i)} (\mathbf{v}_i - \mathbf{v}_j) = \mathbf{0}, \quad \text{for all interior } i \quad (5)$$

这等价于求解线性方程组 $\mathbf{L}\mathbf{V} = \mathbf{B}$ ，其中 $\mathbf{L}$ 为拉普拉斯矩阵， $\mathbf{V}$ 为顶点坐标向量， $\mathbf{B}$ 为边界条件向量。

2 代码实现

2.1 半边数据结构与3D网络结构

在实现中，我们使用OpenMesh库提供的半边数据结构来高效遍历网格元素。半边结构提供了便利操作，快速访问顶点的邻域顶点、高效遍历面和边、便捷的边界检测。

边界检测的关键代码：

```
std::unordered_map<int,int> boundary_next;
for (const auto& heh : halfedge_mesh->halfedges()) {
    if (heh.is_boundary()) {
        boundary_next[heh.from().idx()] = heh.to().idx();
    }
}
```

Listing 1: 边界检测实现

2.2 Param节点实现

2.2.1 固定边界求解极小曲面

在hw5_param.cpp中实现了拉普拉斯方程的构建和求解：

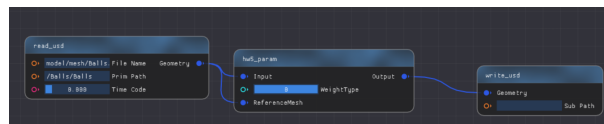


Figure 1: 节点连接图

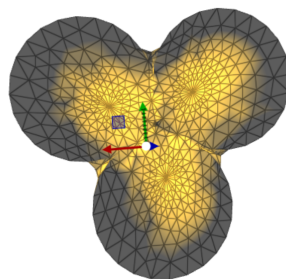


Figure 2: balls效果图

1. 检测内部顶点和边界顶点

2. 构建拉普拉斯矩阵 $\mathbf{L}$
3. 设置边界条件 $\mathbf{B}$
4. 求解线性方程组

求解时遇到了问题与解决方案：求解器选型错误：用对称矩阵求解器（SimplicialLDLT）解非对称矩阵位移钳制破坏解：限制了顶点位移，导致最终坐标不是方程的解

2.2.2 不同权重的具体实现

实现了三种不同的权重策略：

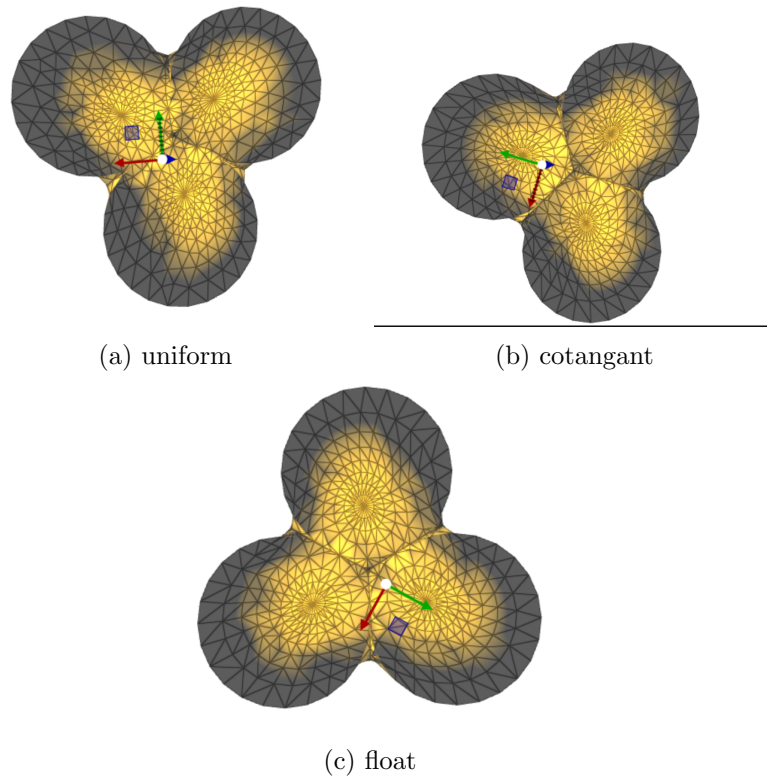


Figure 3: 不同参数化权重效果

- 均匀权重实现：使用顶点度数的倒数
- Cotangent权重实现：基于三角形内角计算
- Floater权重实现：使用平均值坐标公式

实现时容易错：cotangent角度计算，对于顶点 i 的临点 j ， α 和 β 分别表示的是 ij 边的相邻两个三角形的对顶点，而不是 j 处或者 i 处张角，这个容易搞错。不过结合助教在文档中给出的图片可以方便理解；

关键的Floater权重计算函数：

```
auto compute_floater_weights = [&](const auto* mesh_ptr, int
    center_old_idx,
    const std::vector<int>& neighbor_old_indices, std::vector<double>&
        weights) -> bool {
```

```
};
```

Listing 2: Floater

2.3 Boundary Map节点实现

2.3.1 圆形和方形平面映射

在hw5_boundary_map.cpp中实现了边界映射功能:

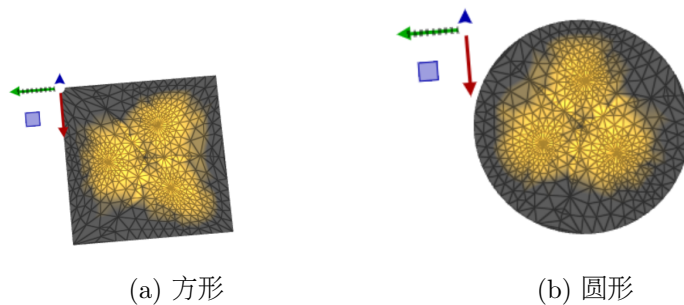


Figure 4: 方形和圆形边界映射

- 边界提取: 通过半边结构识别边界环
- 圆形映射: 将边界点均匀分布到单位圆周上
- 方形映射: 将边界点均匀分布到单位正方形边界上

圆形映射的关键算法:

Algorithm 1 边界圆形映射算法

输入: 边界顶点列表`boundary_vertices`
计算边界长度`total_length`
for 每个边界顶点 v_i **do**
 计算累积弧长`cumulative_length`
 计算角度 $\theta = 2\pi \cdot \text{cumulative_length} / \text{total_length}$
 设置新坐标 $v_i = (\cos(\theta), \sin(\theta))$
end for

3 纹理可视化样例

3.1 多个实例模型的可视化结果

使用测试纹理test.png对参数化结果进行可视化验证:

3.2 不同权重参数化贴图可视化

对比分析不同权重策略对纹理映射质量的影响:

- 均匀权重: 产生较为规则但可能失真的参数化

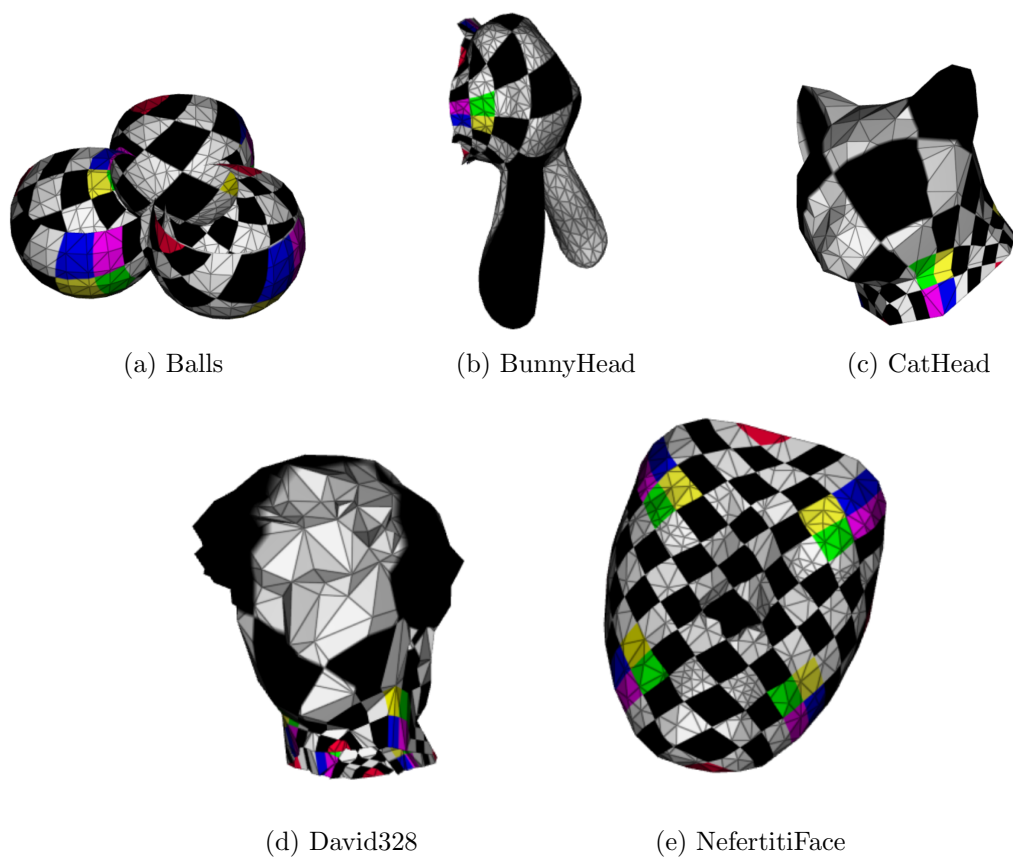


Figure 5: 不同模型可视化效果

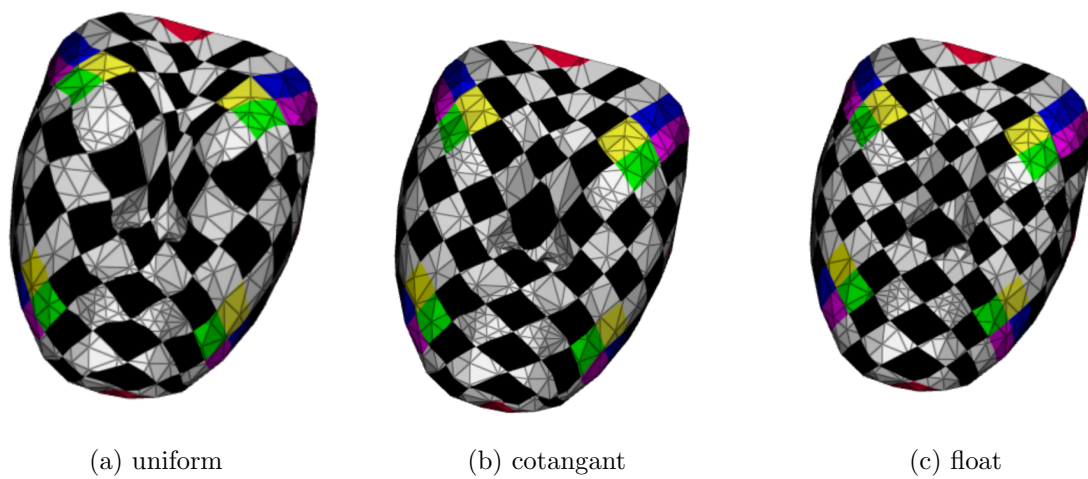


Figure 6: 不同模型效果

- **Cotangent**权重: 保持更好的角度关系
- **Floater**权重: 提供最优的形状保持性

4 实验结果与分析

4.1 稳定性优化措施

在实现过程中，针对数值不稳定问题进行了多项优化：

1. 正则化处理：在拉普拉斯矩阵中加入小的正则化项避免奇异
2. 边界点固定：确保边界条件的准确应用
3. 权重归一化：防止数值漂移

4.2 性能分析

不同权重策略的时间复杂度均为 $O(n)$ ，其中 n 为顶点数量，但Floater权重需要额外的几何计算，因此实际运行时间略长。

4.3 质量评估

通过以下指标评估参数化质量：

- 角度保持性
- 面积变形程度
- 边界映射精度

5 思考题

5.1 高亏格曲面（详细实现文档里面的思考题）

均匀权重、余切权重、Floater 均值坐标权重，本质上都属于Tutte barycentric mapping（重心映射）家族。其理论基石是Tutte 平面嵌入定理：仅当输入是单连通三角网格（亏格 $g=0$ ，仅一个边界环）时，将边界映射到凸区域、内部顶点取邻居凸组合的参数化，才能保证全局无重叠、无翻转的单射结果。

但是（亏格 $g \geq 1$ ，如环面 $g=1$ 、双环面 $g=2$ ，对应“有洞”的曲面）天然不满足这个前提——它无法全局同胚映射到单个平面凸区域，必然存在拓扑奇点或自交。这是所有分析的根本出发点：三种权重都不能解决高亏格曲面的全局参数化问题，仅能在“局部单连通片”或“近似场景”下表现出差异。

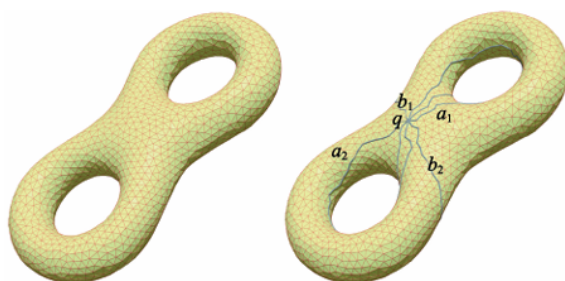


Figure 7: 高亏格曲面与算法切割实例

针对该问题进一步优化方案：切割曲面为单连通片：将高亏格曲面沿若干条边切割，得到多个单连通的子网格，分别用Floater 权重进行参数化，再拼接参数域。这是工业界最常用的方

法。多边界环独立映射：修改边界映射代码，提取所有边界环，将每个洞的边界映射到参数域内的独立凸区域（如同心圆），再进行参数化求解。引入正则化项：在拉普拉斯矩阵中添加小的对角正则化，缓解高亏格下的数值不稳定问题。升级参数化方法：若需要全局无重叠的高亏格参数化，需放弃Tutte 参数化，改用LSCM（最小二乘保形映射）、ABF++（角度基于扁平化）等专门针对高亏格的方法。

5.2 其他权重（思考2）

5.2.1 平均值坐标权重（Mean Value Coordinates）

平均值坐标是重心坐标在任意凸多边形（或凸多面体）上的推广，其离散形式定义如下：

设平面凸多边形顶点为 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \mathbb{R}^2$ ，对多边形内任意点 $\mathbf{x}$ ，其平均值坐标权重 w_i 满足：

$$\mathbf{x} = \sum_{i=1}^n w_i \mathbf{x}_i, \quad \sum_{i=1}^n w_i = 1, \quad w_i > 0.$$

对网格边 (i, j) ，其权重计算公式为：

$$w_{ij} = \frac{\tan(\gamma_{ij}/2) + \tan(\delta_{ij}/2)}{\|\mathbf{x}_i - \mathbf{x}_j\|},$$

其中：- γ_{ij}, δ_{ij} 为边 (i, j) 两侧相邻三角形的对角（见图1）；- $\|\mathbf{x}_i - \mathbf{x}_j\|$ 为顶点 $\mathbf{x}_i$ 与 $\mathbf{x}_j$ 的欧氏距离。

- 非负性：对凸多边形， $w_{ij} > 0$ 恒成立，数值稳定性优异；
- 线性精确性：若 $\mathbf{x}$ 为线性函数，则插值结果精确；
- 凸包性：插值点始终位于顶点凸包内。

网格参数化、自由变形（FFD）、形状插值、骨骼蒙皮。

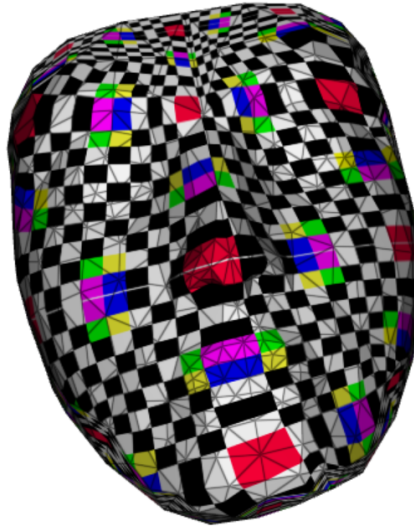


Figure 8: 平均值坐标权重

2.调和权重（Harmonic Weights）

原理：求解拉普拉斯方程 $\Delta u = 0$ ，边界条件为 $u=1$ 在源点， $u=0$ 在其他边界特点：权重分布最平滑，没有局部极值；理论上最优的插值权重；自动适应网格的几何和拓扑；

适用场景：高质量变形、骨骼蒙皮、纹理映射缺点：需要求解线性系统，速度比前几种慢

3. 双调和权重 (Biharmonic Weights)

原理：求解双调和方程 $\Delta^2 u = 0$ 特点：比调和权重更平滑， C^2 连续；对边界条件不敏感；适合需要高阶光滑性的应用；适用场景：高精度变形、形状编辑、动画制作；缺点：线性系统规模更大，求解时间更长；

5.3 曲面微分坐标的定义 (思考1)

曲面的微分坐标（拉普拉斯坐标）是离散微分几何中描述局部形状的核心工具。对于三角网格顶点 v_i ，均匀权重微分坐标定义为：

$$\delta_i = v_i - \frac{1}{\deg(i)} \sum_{j \in N(i)} v_j$$

其中 $N(i)$ 是顶点 i 的一阶邻域顶点集合， $\deg(i)$ 为顶点度数。

这一定义本质是连续拉普拉斯-贝尔特拉米算子的离散近似，度量顶点与邻域质心的偏差，反映曲面局部弯曲程度。其关键性质包括：平移不变性（整体平移时坐标不变）、线性变换协变性（网格变换时坐标同步变换）和局部细节表示（剥离全局位置，仅保留局部特征）。均匀权重仅考虑拓扑，对不规则网格误差较大，更精确的实现可采用余切权重[1]。

5.4 非盘同胚网格的参数化处理 (思考3)

Tutte均匀参数化（重心映射）要求网格与单位圆盘同胚（单连通、单一闭边界），否则会出现参数域重叠。针对不同拓扑类型，处理方法如下：

对于带多个边界的亏格0网格（有洞曲面），可缝合内部边界：将内部洞的边界顶点连接到虚拟顶点，转化为单一边界网格，实现简单但虚拟顶点附近有扭曲[2]。也可固定多边界：外边界固定在单位圆，内部洞边界固定在参数域凸多边形，无需修改拓扑，扭曲更小。

对于高亏格网格（ $g \geq 1$ ），需通过切割转化为盘同胚网格：先计算同调基找到 $2g$ 条切割线，沿切割线切开网格后应用Tutte参数化，最后标记接缝。切割线应尽量短且沿特征线分布，减少视觉影响[3]。非流形网格需先拆分非流形边/顶点，修复拓扑后再参数化。

5.5 均匀参数化的应用 (思考4)

均匀参数化建立了三维网格与二维参数域的一一对应，为几何处理提供基础平台。

基础应用包括纹理映射：将二维纹理通过参数化映射到三维表面，实现视觉增强；网格重采样：在参数域均匀采样后逆映射回三维，将不规则网格转化为规则网格；网格平滑：在参数域进行二维滤波，比三维操作更高效且保留整体形状。

高级应用包括形状变形：在参数域操纵控制顶点，将变形传递到三维，实现自由变形；形状匹配：将不同形状参数化到同一域，转化为二维匹配问题，降低复杂度；孔洞填充：在参数域完成二维填充后映射回三维，效果更自然。此外，均匀参数化常作为复杂参数化方法的初始值，通过优化进一步降低扭曲。

6 与AI交互的经验

具体交互案例在md文档里面。

6.1 agent多文件夹项目读取

考虑想要实现Ruzino的复制粘贴功能，我会把整个项目喂给agent，然后让其根据项目路径分析；但是项目比较庞大，加之我这个trae模型比较弱，找不到位置，需要把指导md也丢给AI让其分析，并自己给出一些已知的代码路径，这样可以一方面降低注意力分散，可以集中在应该寻找的地方，方便后一步分析。这个复制功能牵扯比较多，容易弄坏框架。

正误与优劣分析、评估与修改：我如果不给他提示路径他找不到，需要提供正确路径，并且agent能力有限，并不能找到所有有用代码，需要提示与引导。2.agent修复bug，求解器选择错误根据效果图发现边界优化以及有很多求解有问题，然后需要结合代码分析，查找发现是求

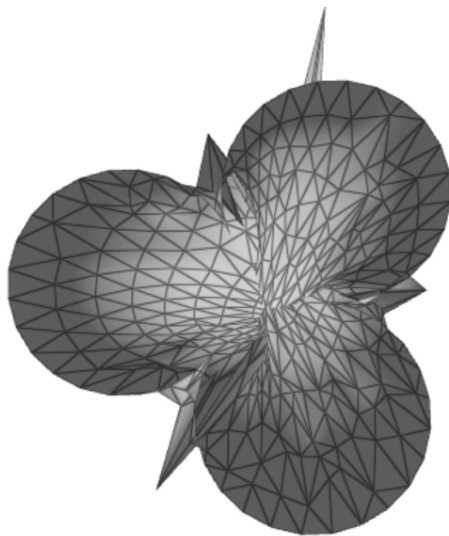


Figure 9: Failed

解器选错了，一个适合对称矩阵的求解器不能求解该问题；

正误与优劣分析、评估与修改：一开始直接选取错误的求解器，唐完了，属于是超级低级错误。但是一开始我也不知道，直到看了结果发现不对.....。

6.2 AI辅助整体的评价与反思

帮了很多忙，但是一开始例如不限于求解去选择失败让我找了好半天（没想到如此低级错误）甚至考虑了其他情况，评价是特别唐。反思是不要全信这个AI，尤其是一些关键的选择！！（可能是我agent比较垃圾）

7 总结与展望

本文成功实现了基于Tutte参数化的曲面参数化算法，验证了不同权重策略的有效性。实验结果表明，Floater权重在形状保持方面表现最佳，而均匀权重计算效率最高。未来工作可考虑扩展到高亏格曲面的参数化以及自适应权重策略的探索。

References

- [1] libigl: Laplace-Beltrami Operator. <https://libigl.github.io/tutorial/#laplace-beltrami-operator>
- [2] libigl: Tutte Embedding. <https://libigl.github.io/tutorial/#tutte-embedding>
- [3] libigl: Mesh Cutting. <https://libigl.github.io/tutorial/#mesh-cutting>